coursera | IBM

Fundamentals of Building AI Agents

Today's goal • 3/3 items

Today's Skill Points 38 XP
See skill progress

Module 1
Foundations of Tool Calling and Chaining

Welcome to the Course
- Course Introduction — Video • 2 min
- Course Overview — Reading • 3 min
- Helpful Tips for Course Completion — Ungraded Plugin • 1 min
- RAG and Agentic AI Professional Certificate Overview — Video • 5 min

Introduction to AI Agents
- What are AI Agents? — Video • 12 min
- Comparing AI System Designs — Reading • 5 min
- When to (and not to) use AI Agents — Reading • 5 min
- Practice Quiz: Introduction to AI Agents — Practice Assignment • Grade: 100%

Getting Started with Tool Calling
- Tool Calling for LLMs — Video • 4 min
- Why AI Needs Tools: From Guessing to Real-World Action — Video • 4 min
- Tools, Agents, and Function Calling in LangChain — Reading • 10 min
- Practice Quiz: Getting Started with Tool Calling — Practice Assignment • Grade: 100%

Building and Orchestrating Tools
- Build Effective AI Tools for Advanced LLMs — Video • 8 min
- Build Intelligent Agents for Dynamic LLM Tool Use — Video • 8 min
- Build a Custom Math Toolkit Agent with LangChain — Video • 6 min
- Lab: Build an AI Math Assistant with LangChain Tool Calling — Ungraded App Item • 45 min
- Popular Built-in Tools in LangChain — Reading • 5 min
- Practice Quiz: Building and Orchestrating Tools — Practice Assignment • Grade: 100%

Module Summary and Evaluation
- Summary and Highlights: Foundations of Tool Calling and Chaining — Reading • 2 min
- Cheat Sheet: Foundations of Function Calling and Chaining — Reading • 20 min
- Graded Quiz: Foundations of Tool Calling and Chaining — Graded Assignment • Grade: 100%
- Reflecting on AI Agents: From Concepts to Capabilities — Dialogue • 15 min

Module 2
LCEL and Manual Tool Calling in LangChain

Introduction to Chaining and LCEL Basics
- LangChain LCEL Chaining Method — Video • 4 min
- Lab: AI Powered Data Analysis with LCEL — Ungraded App Item • 45 min
- Cheat Sheet: LangChain Expression Language (LCEL) — Reading • 5 min
- Practice Quiz: Introduction to Chaining & LCEL Basics — Practice Assignment • 10 min

Manual Tool Calling Basics
- When to Call Tools Manually — Video • 4 min
- Structured Outputs for Tool Calling — Reading • 5 min
- Practice Quiz: Manual Tool Calling Basics — Practice Assignment • 10 min

Parsing and Validating Tool Calls
- Build LLM Agents with Tools — Video • 4 min
- Build Interactive LLM Agents — Video • 5 min
- Lab: Build Interactive LLM Agents with Tools — Ungraded App Item • 1h
- Lab: Build a Tool Calling Agent — Ungraded App Item • 1h
- Practice Quiz: Parsing and Validating Tool Calls — Practice Assignment • 15 min

Module Summary and Evaluation
- Summary and Highlights: Introduction to Chaining and LCEL Basics — Reading • 2 min
- Cheat Sheet: Manual Tool Calling in LangChain — Ungraded Plugin • 20 min
- Graded Quiz: Manual Tool Calling in LangChain — Graded Assignment • 21 min

Module 3
Using Built-in Agents in LangChain

Structured Outputs for Tool Calling

3 XP • Prototyping and Strategic Evaluation of Generative AI

0:03 / 4:39 1x

Learning Objectives

By the end of this reading, you will be able to:

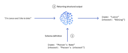
- Define structured output schemas using JSON-like formats and Pydantic models to enforce data consistency in LLM-generated responses.
- Implement tool calling and JSON mode to generate and validate structured outputs for use in databases, APIs, and application workflows.

This reading explores the use of structured outputs in large language model (LLM) applications, crucial for producing consistent, schema-compliant responses. Learners will discover how to implement output schemas using Pydantic models, tool calling, and JSON mode to support reliable database integration, API workflows, and data validation across various applications.

While many LLM applications respond to users in natural language, there are scenarios where we need models to output in a structured format. Structured outputs allow you to:

- Store model responses in databases with consistent formats
- Process outputs programmatically in your application
- Ensure responses include specific fields in a predictable structure
- Validate output against a defined schema

The process involves two main steps, as illustrated in the image:



1. **Schema Definition:** Defining the structure that we want the output to have.
2. **Returning structured output:** Having the model generate output conforming to the schema.

For example, given an input like *"I'm Lance and I like to bike,"* we can define a schema with fields for *"name"* and *"interest"*. The LLM processes this input against the schema and returns a structured output: {name: "Lance", interest: "Biking"}.

Schema definition

The schema defines the structure your model's output should follow. In LangChain, you have several options for defining schemas:

1. **JSON like structures:** The simplest format is a JSON-like structure, represented in Python as a dictionary or list.
2. **Pydantic models:** Pydantic is particularly useful for defining structured output schemas because it offers type hints and validation. Pydantic models offer several advantages as well, such as:
   - Type validation
   - Clear field descriptions
   - Built-in documentation
   - Integration with LangChain's tooling

Returning structured output

There are two main approaches to ensure models return structured outputs:

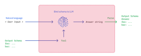
1. Tool calling

- Creating a model
- Binding the ResponseFormatter schema as a tool to the model
- Invoking the model with a user query
- Extracting the tool call arguments as a dictionary
- Optionally, parsing the dictionary into a pydantic object

2. JSON mode

- Requires using a model that supports JSON mode
- Uses the with_structured_output method with the "json_mode" parameter
- Returns results already formatted as a Python dictionary

Structured Output Method: with_structured_output()



As shown in the image, LangChain provides a helper method with_structured_output() that streamlines the entire process:

1. The output schema is provided as a tool to the LLM
2. User input is processed by the LLM with this schema in mind
3. The LLM generates an answer string
4. This output is parsed into the structured schema format

This workflow handles several challenges automatically:

- Binding the schema to the model as a tool
- Instructing the model to always use the tool
- Parsing the output back to the specified schema

Use cases for structured outputs

- **Database storage:** Format responses for direct storage in databases
- **API integration:** Structure outputs to match API requirements
- **UI components:** Format data for display in specific UI elements
- **Multi-step workflows:** Break down complex tasks into structured steps
- **Data extraction:** Extract specific information from unstructured text

Summary

In this reading, you learned that:

1. **Structured outputs enable precision:** LLMs can produce responses in predictable formats, essential for database storage, API integration, and automating workflows.
2. **Schemas define output expectations:** Using JSON-like structures or Pydantic models helps enforce required fields and validate response data programmatically.
3. **Tool calling for schema enforcement:** Binding a schema as a "tool" to a model ensures outputs conform exactly to predefined formats, enabling robust application logic.
4. **JSON mode for reliable outputs:** Models supporting JSON mode can generate structured outputs automatically, reducing the need for post-processing.
5. **LangChain's with_structured_output() simplifies the process:** This helper method streamlines schema binding, model invocation, and output parsing, making structured responses easier to implement.

Author(s)

IBM Skills Network Team

Mark as completed

Like Dislike Report an issue